Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Тип практики: Преддипломная практика

Название предприятия: НУК ИУ МГТУ им. Н.Э.Баумана

Студент: группа ИУ8-124-2026 л.д. 20У474
Железцов Никита Владимирович _____ (подпись, дата)

Руководитель от предприятия:
доцент кафедры ИУ8 Медведев Николай Викторович _____ (подпись, дата)

Руководитель ВКР:
доцент кафедры ИУ8 Медведев Николай Викторович _____ (подпись, дата)

Руководитель от кафедры:
доцент кафедры ИУ8 Зайцева Анастасия Владленовна _____ (подпись, дата)

Оценка: _____

Москва, 2026

РЕФЕРАТ

Отчёт содержит 33 стр., 7 рис., 4 источн., 1 прил.

Работа посвящена верификации соответствия трасс исполнения программы формальной спецификации на языке TLA+.

Одним из возможных способов решения данной проблемы является валидация трасс исполнения программы относительно формальной спецификации, методика которой предложена в данной работе.

В работе предложена методика, основанная на инструментировании программной реализации, построении трассы наблюдаемых шагов исполнения и её интерпретации как системы ограничений на переходы формальной модели. Сформулирована математическая постановка задачи в терминах системы переходов, трассы наблюдений и предиката согласованности шага. Для оценки степени согласованности трассы с моделью введён показатель качества, определяемый длиной максимального согласованного префикса трассы. Предложено решение задачи на основе последовательного вычисления множеств достижимых состояний, согласованных с префиксами трассы, и получен алгоритм нахождения максимального согласованного префикса.

Практическая реализация предложенного подхода выполнена средствами TLA+ и TLC на примере спецификации протокола двухфазной фиксации транзакции.

Ключевые слова: TLA+, TLC, формальная верификация, система переходов, трасса исполнения, распределённые системы.

СОДЕРЖАНИЕ

РЕФЕРАТ . . . 3
ВВЕДЕНИЕ . . . 5
ОСНОВНАЯ ЧАСТЬ . . . 6
1 Характеристика организации . . . 6
2 Методика верификации . . . 7
3 Математическая постановка задачи . . . 8
3.1 Формализация модели и ее поведения . . . 8
3.2 Формализация трассы программы . . . 9
3.3 Согласованность трассы с моделью . . . 10
3.4 Показатель качества согласованности . . . 12
3.5 Итоговая математическая постановка задачи . . . 13
4 Обоснование решения математической постановки задачи . . . 14
4.1 Основная идея решения . . . 14
4.2 Рекуррентное построение множеств D_i . . . 15
4.3 Корректность построения множеств D_i . . . 16
4.4 Нахождение оптимального значения показателя качества . . . 18
4.5 Критерий полной совместимости трассы с моделью . . . 18
4.6 Алгоритм решения . . . 19
5 Реализация задачи и решения средствами TLA+ и TLC . . . 22
5.1 Общая схема реализации . . . 22
5.2 Соответствие модели и TLA+ спецификации . . . 22
5.3 Представление трассы программы . . . 24
5.4 Реализация предиката согласованности шага . . . 25
5.5 Ограниченная спецификация как реализация префиксной совместимости . . . 27
5.6 Соответствие рекуррентных множеств D_i и работы TLC . . . 28
5.7 Критерий успешной проверки в TLC . . . 29
5.8 Особенности и ограничения рассматриваемой реализации . . . 30
ЗАКЛЮЧЕНИЕ . . . 31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 32
ПРИЛОЖЕНИЕ А Методика верификации . . . 33

ВВЕДЕНИЕ

Формальные спецификации применяются для описания допустимого поведения сложных программных систем, в особенности распределённых алгоритмов, для которых характерны недетерминизм, конкурентное взаимодействие и большое число возможных сценариев выполнения. Одним из распространённых средств такого описания является язык TLA+, позволяющий задавать систему в терминах состояний и переходов, а также проверять свойства модели средствами модельной проверки.

Однако наличие формальной спецификации само по себе не гарантирует её актуальность по отношению к реальной реализации. В процессе разработки исходный код системы изменяется, дополняется новыми возможностями и оптимизируется, тогда как формальная модель нередко обновляется с запозданием или не обновляется вовсе. В результате возникает риск расхождения между фактическим поведением программы и поведением, допускаемым спецификацией. Подобное расхождение снижает ценность формальной модели как инструмента анализа и верификации.

Одним из возможных способов решения данной проблемы является валидация трасс исполнения программы относительно формальной спецификации, методика которой предложена в данной работе.

Целью работы является формализация математической постановки задачи верификации соответствия трасс исполнения программы и формальной спецификации на языке TLA+ и обоснование её решения.

Место прохождения практики – Научно-учебный комплекс «Информатика и системы управления» МГТУ им. Н.Э. Баумана. Период прохождения практики – с 9 февраля 2026 года по 10 марта 2026 года.

В задачи прохождения практики входило:

а) Сформулировать математическую постановку задачи верификации соответствия TLA+ спецификации и исходного кода;

б) Предложить возможные методы решения поставленной задачи.

Структурно работа состоит из разделов, посвящённых описанию методики верификации, математической постановке задачи, обоснованию её решения и реализации полученных результатов средствами TLA+ и TLC.

ОСНОВНАЯ ЧАСТЬ

1 Характеристика организации

Научно-учебный комплекс «Информатика и системы управления» [1] (далее — НУК ИУ) является самостоятельным структурным подразделением МГТУ им. Н.Э. Баумана (далее — Университет). Полное наименование НУК ИУ на английском языке Scientific Educational Complex «Informatics and Control Systems» Bauman Moscow State Technical University. Сокращённое — SEC ICS BMSTU.

НУК ИУ возглавляет руководитель НУК ИУ д.т.н., профессор и заведующий кафедрой ИУ6 («Компьютерные системы и сети») Пролетарский А.В., который непосредственно подчиняется действующему ректору Университета к.т.н. Гордину М.В..

Цель деятельности НУК ИУ: обеспечение образовательного и научного процессов в соответствии с Уставом Университета. К основным задачам подразделения относят:

- Подготовку бакалавров, специалистов, магистров, научных и научно-педагогических кадров высшей квалификации в соответствии с государственными лицензиями, выданными Университету.
- Выполнение фундаментальных, поисковых и прикладных научных исследований, проведение опытно-конструкторских работ и производство опытных образцов перспективной техники и технологий.
- Написание и издание учебников, учебных пособий и монографий.
- Развитие научных, педагогических и инженерных школ НУК ИУ.
- Разработку и внедрение прогрессивных форм, методов и средств подготовки специалистов.
- Повышение квалификации и переподготовка научно-педагогических кадров технических и экономических учебных заведений, а также специалистов предприятий с высшим образованием.
- Развитие материально-технической базы НУК ИУ.
- Организацию научной работы студентов.
- Социальную поддержку и защиту преподавателей, научного и отвечающих требованиям подготовки вспомогательного персонала.

2 Методика верификации

Предлагаемая методика верификации основана на сопоставлении трасс исполнения программы с формальной спецификацией, написанной на языке TLA+. Схема подхода представлена в Приложении А.

Исходными артефактами являются программная реализация алгоритма и формальная спецификация этого алгоритма на языке TLA+. Спецификация задаёт множество допустимых состояний системы и переходов между ними, однако не содержит сведений о конкретной реализации. Реализация, в свою очередь, выполняется в распределённой среде, где возможны различные порядки доставки сообщений и недетерминированные сценарии выполнения.

Для сопоставления поведения реализации со спецификацией используется инструментирование кода. В реализацию внедряется библиотека трассировки, которая фиксирует события, соответствующие переходам спецификации, а также изменения переменных спецификации. При выполнении системы каждый процесс (координатор и участники) формирует собственную трассу в формате JSON, содержащую последовательность операций обновления логических переменных модели. Полученные локальные трассы агрегируются в единый глобальный файл, отражающий наблюдаемую последовательность изменений состояния всей распределённой системы. Таким образом, формируется абстрактное представление исполнения, выраженное в терминах переменных TLA+, а не в терминах низкоуровневых операций кода.

Для проверки корректности трассы используется дополнительная спецификация, построенная на основе исходной TLA+ спецификации. Она расширяет её таким образом, чтобы поведение модели было ограничено наблюдаемой последовательностью изменений. Иными словами, трасса интерпретируется как частичное ограничение на допустимые переходы модели.

Далее средство модельной проверки TLC используется для проверки существования поведения спецификации, согласованного с данной трассой. Если TLC подтверждает корректность, это означает, что наблюдаемое исполнение допустимо спецификацией. В противном случае обнаруживается расхождение между реализацией и моделью, и TLC предоставляет информацию, позволяющую локализовать проблему.

3 Математическая постановка задачи

В данном разделе задача верификации соответствия формальной спецификации и исходного кода формализуется в терминах системы переходов, трассы наблюдений и предиката согласованности шага. На этой основе задаётся строгий критерий согласованности трассы с моделью как условие существования допустимого поведения модели, совместимого с наблюдаемой трассой.

3.1 Формализация модели и ее поведения

Для формализации модели (спецификации) программы используется понятие *системы переходов* (transition system) [2] с выделенным множеством начальных состояний.

Определение 3.1. Системой переходов называется тройка

$$\mathcal{M} = (X, X_0, R), \tag{1}$$

где

– X — множество состояний;
– $X_0 \subseteq X$ — множество начальных состояний;
– $R \subseteq X \times X$ — отношение переходов.

Элемент $x \in X$ интерпретируется как полное абстрактное описание состояния системы в некоторый момент её выполнения. Условие $(x, y) \in R$ означает, что из состояния x система может перейти в состояние y за один допустимый шаг. Иначе говоря, отношение R задаёт все допустимые одношаговые изменения состояния системы.

Определение 3.2. Конечным поведением системы переходов $\mathcal{M}$ длины $n \in \mathbb{N}_0$ называется последовательность состояний

$$\pi_n = (x_0, x_1, \ldots, x_n) \in X^{n+1}, \tag{2}$$

для которой выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R.$$

Первое условие означает, что поведение начинается из некоторого допустимого начального состояния. Второе условие означает, что каждый следующий элемент последовательности получается из предыдущего допустимым переходом системы. Таким образом, конечное поведение есть путь в ориентированном графе, порождаемом системой переходов, начинающийся в одной из вершин из множества X_0.

Определение 3.3. Для каждого $n \in \mathbb{N}_0$ обозначим через $\mathrm{B}_n(\mathcal{M})$ множество всех конечных поведений системы переходов $\mathcal{M}$ длины n:

$$\mathrm{B}_n(\mathcal{M}) = \left\{(x_0, \ldots, x_n) \in X^{n+1} : x_0 \in X_0,\ \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R\right\}.$$

Тогда множество всех конечных поведений системы переходов $\mathcal{M}$ обозначается через $\mathrm{B}(\mathcal{M})$ и определяется равенством

$$\mathrm{B}(\mathcal{M}) = \bigcup_{n \in \mathbb{N}_0} \mathrm{B}_n(\mathcal{M}). \tag{3}$$

Множество $\mathrm{B}(\mathcal{M})$ содержит все конечные пути в системе переходов, начинающиеся из допустимых начальных состояний. Тем самым оно полностью определяет допустимое наблюдаемое поведение модели на конечных префиксах.

Если некоторое исполнение реализации не может быть согласовано ни с одним элементом множества $\mathrm{B}(\mathcal{M})$, то это означает, что наблюдаемое поведение не принадлежит множеству допустимых поведений, задаваемых абстрактной моделью. Следовательно, в этом случае фиксируется расхождение между реализацией и спецификацией.

3.2 Формализация трассы программы

Реальная программа в процессе выполнения порождает наблюдаемую трассу. В рассматриваемой постановке предполагается, что трасса не содержит отдельного наблюдения начального состояния системы. Начальное состояние выбирается из множества начальных состояний модели, а трасса фиксирует только наблюдения, относящиеся к последующим шагам выполнения программы.

При этом в рамках рассматриваемой постановки предполагается, что каждой записи трассы соответствует ровно один переход поведения модели. Иначе говоря, трасса и поведение модели сопоставляются *пошагово*.

Определение 3.4. Пусть Σ — множество всех возможных наблюдений. Тогда наблюдаемая трасса длины $m \in \mathbb{N}_0$ задаётся последовательностью

$$\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m. \tag{4}$$

Каждый элемент σ_i интерпретируется как наблюдение, относящееся к i-му шагу выполнения программы. Таким образом, поведение модели задаётся последовательностью состояний $(x_0, x_1, \ldots, x_m)$, тогда как наблюдаемая трасса задаётся последовательностью наблюдений $(\sigma_1, \sigma_2, \ldots, \sigma_m)$, в которой каждый элемент σ_i относится к переходу из состояния x_{i-1} в x_i.

Определение 3.5. Для каждого $k \in \{0, \ldots, m\}$ префиксом трассы длины k называется последовательность

$$\tau^{(k)} = (\sigma_1, \sigma_2, \ldots, \sigma_k) \in \Sigma^k,$$

где при $k = 0$ понимается пустая последовательность.

3.3 Согласованность трассы с моделью

После введения понятий модели и трассы можно строго определить, что означает их согласованность.

В общем случае трасса программы представляет собой последовательность неполных изменений состояния системы: обычно она содержит лишь частичную информацию о каждом шаге выполнения, существенную для верификации, например сведения о произошедшем событии и об изменениях отдельных компонент состояния. Поэтому необходимо формально задать, когда переход считается согласованным с данным наблюдением.

Определение 3.6. Предикатом согласованности наблюдаемого шага называется отображение

$$C : \Sigma \times X \times X \to \{0, 1\}. \tag{5}$$

Значение $C(\sigma, x, y) = 1$ означает, что наблюдение σ согласованно с переходом из состояния x в состояние y.

Предикат C задаёт способ интерпретации трассы в терминах абстрактной модели. Его конкретный вид определяется составом данных, записываемых в трассу. Например, если наблюдение фиксирует имя произошедшего события и значения некоторых изменяемых компонент состояния, то согласованность означает, что рассматриваемый переход принадлежит классу переходов, соответствующему данному событию, и приводит к зафиксированным изменениям состояния. Если же наблюдение фиксирует значения лишь некоторых компонент следующего состояния, то согласованность означает совпадение именно этих компонент, тогда как остальные компоненты могут выбираться произвольно при условии допустимости перехода.

Определение 3.7. Пусть дан префикс трассы $\tau^{(k)} = (\sigma_1, \ldots, \sigma_k)$, где $k \in \{0, \ldots, m\}$. Тогда поведение модели

$$\pi_k = (x_0, x_1, \ldots, x_k) \in X^{k+1}$$

называется *согласованным с префиксом трассы* $\tau^{(k)}$, если выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{1, \ldots, k\} : C(\sigma_i, x_{i-1}, x_i) = 1.$$

В рамках данной постановки не рассматриваются сопоставления, при которых одному элементу трассы соответствует несколько переходов модели или, наоборот, одному переходу модели соответствует несколько элементов трассы. Такое ограничение соответствует используемой схеме проверки, в которой каждая запись трассы обрабатывается за один шаг ограниченной спецификации.

Определение 3.8. Для каждого $k \in \{0, \ldots, m\}$ обозначим через $\mathrm{M}_k(\tau)$ множество всех поведений длины k, согласованных с префиксом трассы $\tau^{(k)}$:

$$\mathrm{M}_k(\tau) = \left\{(x_0, \ldots, x_k) \in X^{k+1} : x_0 \in X_0,\ \forall i \in \{1, \ldots, k\} : C(\sigma_i, x_{i-1}, x_i) = 1\right\}. \tag{6}$$

Определение 3.9. Префикс трассы $\tau^{(k)}$, где $k \in \{0, \ldots, m\}$, называется *совместимым* с моделью $\mathcal{M}$, если

$$\mathrm{B}_k(\mathcal{M}) \cap \mathrm{M}_k(\tau) \neq \varnothing. \tag{7}$$

Иными словами, совместимость префикса трассы с моделью означает существование хотя бы одного поведения длины k, которое допускается моделью и одновременно объясняет первые k наблюдений трассы.

В частности, вся трасса $\tau = (\sigma_1, \ldots, \sigma_m)$ совместима с моделью $\mathcal{M}$ тогда и только тогда, когда её префикс полной длины m совместим с моделью, то есть выполнено условие

$$\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau) \neq \varnothing. \tag{8}$$

3.4 Показатель качества согласованности

Для введения количественной оценки согласованности трассы с моделью рассмотрим множество всех допустимых длин согласованных префиксов.

Определение 3.10. Множеством допустимых длин согласованных префиксов называется множество

$$K(\mathcal{M}, \tau) = \{k \in \{0, \ldots, m\} : \mathrm{B}_k(\mathcal{M}) \cap \mathrm{M}_k(\tau) \neq \varnothing\}. \tag{9}$$

Иначе говоря, число k принадлежит множеству $K(\mathcal{M}, \tau)$ тогда и только тогда, когда существует допустимое поведение модели длины k, согласованное с префиксом трассы длины k.

Определение 3.11. Длиной максимального согласованного префикса трассы называется величина

$$k^*(\mathcal{M}, \tau) = \max K(\mathcal{M}, \tau). \tag{10}$$

Величина $k^*(\mathcal{M}, \tau)$ показывает, сколько первых наблюдений трассы можно согласовать с моделью. Если $k^*(\mathcal{M}, \tau) = m$, то вся трасса согласована с моделью. Если же $k^*(\mathcal{M}, \tau) < m$, то согласованным является только собственный префикс трассы.

Определение 3.12. Показателем качества согласованности трассы с моделью называется функция

$$q(k) = \frac{k}{m}, \qquad k \in \{0, \ldots, m\}, \qquad m \geq 1. \tag{11}$$

Тогда оптимальное значение показателя качества определяется равенством

$$Q(\mathcal{M}, \tau) = q(k^*(\mathcal{M}, \tau)) = \frac{k^*(\mathcal{M}, \tau)}{m}. \tag{12}$$

3.5 Итоговая математическая постановка задачи

Пусть $\mathcal{M} = (X, X_0, R)$ — система переходов, где X — множество состояний, $X_0 \subseteq X$ — множество начальных состояний, $R \subseteq X \times X$ — отношение переходов.

$\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$ — трасса программы, где Σ — множество наблюдений, а σ_i представляет собой наблюдение изменения состояния при переходе из $i-1$ в i.

$C : \Sigma \times X \times X \to \{0, 1\}$ — предикат согласованности шага, в котором значение $C(\sigma, x, y) = 1$ означает, что наблюдение σ согласованно с переходом из состояния x в состояние y.

В качестве переменной оптимизации рассматривается длина согласованного префикса трассы $k \in \{0, \ldots, m\}$. Для каждого допустимого значения k введён показатель качества согласованности $q(k) = \frac{k}{m}$, где $m \geq 1$.

Тогда задача заключается в нахождении такого значения длины префикса трассы k, для которого показатель качества максимален и для которого существует допустимое моделью $\mathcal{M}$ поведение длины k, согласованное с префиксом трассы $\tau^{(k)}$. Иначе говоря, требуется решить задачу оптимизации по k с одним показателем качества и системой ограничений:

$$\begin{cases} q(k) = \dfrac{k}{m} \to \max\limits_k, \\ k \in \{0, \ldots, m\}, \\ \exists (x_0, \ldots, x_k) \in X^{k+1} : \\ \qquad x_0 \in X_0, \\ \qquad \forall i \in \{0, \ldots, k-1\} : (x_i, x_{i+1}) \in R, \\ \qquad \forall i \in \{1, \ldots, k\} : C(\sigma_i, x_{i-1}, x_i) = 1. \end{cases} \tag{13}$$

При этом исходная задача верификации соответствия трассы и модели является связанной с (13) задачей разрешимости: требуется определить, достигается ли идеальное значение показателя качества, то есть истинно ли условие $q(k^*) = 1$. Это условие эквивалентно равенству $k^* = m$, а следовательно, и условию полной согласованности трассы с моделью: $\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau) \neq \varnothing$ (по определению 3.10 и 3.11).

4 Обоснование решения математической постановки задачи

В предыдущем разделе математическая постановка задачи была сформулирована как оптимизационная задача максимизации показателя качества $q(k) = \frac{k}{m}$, где k — длина согласованного префикса трассы $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m)$. Ограничения задачи требуют, чтобы для выбранного значения k существовало допустимое поведение модели длины k, согласованное с префиксом трассы $\tau^{(k)}$.

Следовательно, задача состоит в нахождении максимально длинного префикса наблюдаемой трассы, который может быть объяснён допустимым поведением модели. В настоящем разделе предлагается решение этой задачи и доказывается его корректность.

4.1 Основная идея решения

Из формулировки задачи непосредственно следует естественная идея её решения. Поскольку трасса состоит из конечного числа наблюдений, поиск максимального согласованного префикса можно выполнять последовательно, по мере увеличения его длины.

Для этого необходимо:

а) определить множество состояний, достижимых после согласования первого наблюдения σ_1;

б) затем определить множество состояний, достижимых после согласования первых двух наблюдений (σ_1, σ_2);

в) продолжать этот процесс вплоть до последнего наблюдения σ_m;

г) найти наибольший индекс k, для которого после согласования первых k наблюдений остаётся хотя бы одно достижимое состояние модели.

Если после обработки всех m наблюдений множество достижимых состояний непусто, то трасса полностью совместима с моделью, а значение показателя качества равно единице. Если же на некотором шаге множество достижимых состояний становится пустым, то дальнейшее продолжение невозможно, а значит, совместим лишь некоторый собственный префикс трассы.

Тем самым задача максимизации показателя качества сводится к последовательному вычислению достижимых множеств состояний.

4.2 Рекуррентное построение множеств D_i

Для формализации указанной идеи введём последовательность множеств

$$D_0, D_1, \ldots, D_m \subseteq X,$$

где каждое множество D_i интерпретируется как множество всех состояний модели, достижимых после согласования первых i наблюдений трассы.

Поскольку трасса не содержит отдельного наблюдения начального состояния, исходное множество определяется только множеством начальных состояний модели.

Определение 4.1. Начальное множество достижимых согласованных состояний определяется равенством

$$D_0 = X_0. \tag{14}$$

Это определение означает, что до обработки трассы допустимыми считаются все начальные состояния модели.

Далее, для каждого $i \in \{1, \ldots, m\}$ определим множество D_i рекуррентно.

Определение 4.2. Для каждого $i \in \{1, \ldots, m\}$ множество достижимых согласованных состояний после обработки первых i наблюдений определяется равенством

$$D_i = \{y \in X : \exists x \in D_{i-1} : (x, y) \in R \wedge C(\sigma_i, x, y) = 1\}. \tag{15}$$

Смысл этого определения состоит в следующем. Состояние y включается в множество D_i тогда и только тогда, когда существует хотя бы одно состояние $x \in D_{i-1}$, уже достижимое после согласования первых $i-1$ наблюдений, такое, что переход из x в y:

- является допустимым переходом модели, то есть $(x, y) \in R$;
- согласован с i-м наблюдением σ_i, то есть $C(\sigma_i, x, y) = 1$.

Таким образом, определение 4.2 отражает принцип пошагового распространения ограничений: каждое следующее наблюдение применяется только к тем состояниям, которые уже достижимы после обработки предыдущего префикса трассы. Это означает, что множество D_0 описывает все возможные исходные положения системы, множество D_1 описывает все состояния, в которые система может перейти после одного шага, согласованного с σ_1 и т.д.

4.3 Корректность построения множеств D_i

Для обоснования корректности предложенного решения необходимо доказать, что множество D_i действительно имеет именно тот смысл, который ему приписан, а именно: содержит в точности те состояния, которые достижимы после согласования первых i наблюдений трассы.

Утверждение 1. Для любого индекса $i \in \{0, \ldots, m\}$ и любого состояния $x \in X$ следующие утверждения эквивалентны:

а) $x \in D_i$;

б) существует последовательность состояний

$$
\begin{aligned}
&(x_0, x_1, \ldots, x_i) \in X^{i+1} : \\
&x_0 \in X_0, \\
&x_i = x, \\
&\forall k \in \{0, \ldots, i-1\} : \ (x_k, x_{k+1}) \in R, \\
&\forall k \in \{1, \ldots, i\} : \ C(\sigma_k, x_{k-1}, x_k) = 1.
\end{aligned}
$$

Иначе говоря, состояние x принадлежит множеству D_i тогда и только тогда, когда существует путь длины i, начинающийся в допустимом начальном состоянии, допустимый для модели и согласованный с первыми i наблюдениями трассы, который оканчивается в состоянии x.

Доказательство. Доказательство проводится по индукции по i.

База индукции. Пусть $i = 0$. Тогда по определению 4.1 $D_0 = X_0$. Следовательно, для любого $x \in X$ условие $x \in D_0$ эквивалентно условию $x \in X_0$. Это, в свою очередь, эквивалентно существованию последовательности длины 1, состоящей из единственного состояния (x) и удовлетворяющей условиям пункта б. База доказана.

Переход индукции. Пусть утверждение верно для некоторого $i - 1 \in \{0, \ldots, m-1\}$. Докажем его для индекса i.

Необходимость. Пусть $x \in D_i$. Тогда по определению 4.2

$$\exists y \in D_{i-1} : (y, x) \in R \wedge C(\sigma_i, y, x) = 1.$$

По предположению индукции из условия $y \in D_{i-1}$ следует существование последовательности состояний $(x_0, x_1, \ldots, x_{i-1}) \in X^i$, для которой выполнены условия

$$\begin{aligned}
&x_0 \in X_0,\\
&x_{i-1} = y,\\
&\forall k \in \{0, \ldots, i-2\} : (x_k, x_{k+1}) \in R,\\
&\forall k \in \{1, \ldots, i-1\} : C(\sigma_k, x_{k-1}, x_k) = 1.
\end{aligned}$$

Добавляя к этой последовательности состояние x, получаем последовательность $(x_0, x_1, \ldots, x_{i-1}, x) \in X^{i+1}$, для которой дополнительно выполнены условия

$$(x_{i-1}, x) \in R, \qquad C(\sigma_i, x_{i-1}, x) = 1.$$

Следовательно, существует последовательность состояний, удовлетворяющая условиям пункта б.

Достаточность. Пусть существует последовательность $(x_0, x_1, \ldots, x_i) \in X^{i+1}$, для которой выполнены условия

$$\begin{aligned}
&x_0 \in X_0,\\
&x_i = x,\\
&\forall k \in \{0, \ldots, i-1\} : (x_k, x_{k+1}) \in R,\\
&\forall k \in \{1, \ldots, i\} : C(\sigma_k, x_{k-1}, x_k) = 1.
\end{aligned}$$

Тогда её префикс $(x_0, x_1, \ldots, x_{i-1})$ удовлетворяет условиям пункта б для индекса $i-1$, а потому по предположению индукции $x_{i-1} \in D_{i-1}$. Из условий

$$(x_{i-1}, x_i) \in R \qquad \text{и} \qquad C(\sigma_i, x_{i-1}, x_i) = 1$$

и определения 4.2 получаем $x_i \in D_i$. Так как $x_i = x$, то $x \in D_i$.

Тем самым обе импликации доказаны, следовательно, утверждения а и б эквивалентны. По принципу математической индукции утверждение доказано для всех $i \in \{0, \ldots, m\}$. $\square$

4.4 Нахождение оптимального значения показателя качества

Из утверждения 1 непосредственно следует, что условие $D_i \neq \varnothing$ эквивалентно существованию допустимого поведения модели, согласованного с префиксом трассы длины i. Следовательно, длина максимального согласованного префикса может быть найдена по формуле

$$k^*(\mathcal{M}, \tau) = \max \{i \in \{0, \ldots, m\} : D_i \neq \varnothing\}. \tag{16}$$

Следовательно, решение оптимизационной задачи полностью сводится к последовательному вычислению множеств D_i и последующему поиску наибольшего индекса, для которого соответствующее множество непусто.

Теорема 4.1. Оптимальное значение показателя качества согласованности трассы с моделью определяется формулой

$$Q(\mathcal{M}, \tau) = \frac{1}{m} \max \{i \in \{0, \ldots, m\} : D_i \neq \varnothing\}.$$

Доказательство. По определению показателя качества

$$Q(\mathcal{M}, \tau) = \frac{k^*(\mathcal{M}, \tau)}{m},$$

где $k^*(\mathcal{M}, \tau)$ есть длина максимального префикса трассы, для которого существует допустимое поведение модели, согласованное с этим префиксом. Согласно утверждению 1, существование такого поведения для префикса длины i эквивалентно непустоте множества D_i. Следовательно,

$$k^*(\mathcal{M}, \tau) = \max \{i \in \{0, \ldots, m\} : D_i \neq \varnothing\},$$

откуда непосредственно следует требуемая формула. □

4.5 Критерий полной совместимости трассы с моделью

Из доказанного индуктивного инварианта непосредственно вытекает критерий решения связанной задачи разрешимости.

Теорема 4.2. Трасса программы $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m)$ совместима с моделью $\mathcal{M}$ тогда и только тогда, когда $D_m \neq \varnothing$. Эквивалентно, $Q(\mathcal{M}, \tau) = 1$.

Доказательство. По определению полной совместимости трассы с моделью требуется выполнение условия

$$\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau) \neq \varnothing.$$

Это означает, что существует последовательность состояний

$$(x_0, x_1, \ldots, x_m) \in X^{m+1},$$

для которой выполнены условия

$$\begin{aligned}
&x_0 \in X_0,\\
&\forall i \in \{0, \ldots, m-1\} : (x_i, x_{i+1}) \in R,\\
&\forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1.
\end{aligned}$$

Согласно утверждению 1, существование такой последовательности эквивалентно существованию состояния $x_m \in D_m$. Последнее, в свою очередь, эквивалентно условию $D_m \neq \varnothing$.

Далее, условие $D_m \neq \varnothing$ означает, что максимальный согласованный префикс имеет длину не меньше m. Так как по определению $k^*(\mathcal{M}, \tau) \leq m$, получаем $k^*(\mathcal{M}, \tau) = m$. Следовательно,

$$Q(\mathcal{M}, \tau) = \frac{k^*(\mathcal{M}, \tau)}{m} = 1.$$

Обратная импликация доказывается аналогично. □

Тем самым задача разрешимости оказывается частным случаем оптимизационной постановки: необходимо определить, достигается ли наилучшее значение показателя качества.

4.6 Алгоритм решения

Из построения множеств D_i непосредственно следует алгоритм решения как оптимизационной задачи, так и связанной с ней задачи разрешимости. Алгоритм последовательно вычисляет множества достижимых согласованных состояний, находит максимальный индекс k^*, для которого соответствующее множество непусто, и на этой основе вычисляет значение показателя качества.

Алгоритм 1 — Нахождение максимального согласованного префикса трассы

Исходные параметры: система переходов $\mathcal{M} = (X, X_0, R)$; трасса $\tau = (\sigma_1, \ldots, \sigma_m) \in \Sigma^m$; предикат согласованности $C : \Sigma \times X \times X \to \{0, 1\}$

Результат: значение $k^*(\mathcal{M}, \tau)$, значение $Q(\mathcal{M}, \tau)$ и ответ о полной совместимости трассы

$D_0 \leftarrow X_0$;
$k^* \leftarrow 0$;
для каждого $i \in \{1, \ldots, m\}$ **выполнять**
  $D_i \leftarrow \{y \in X : \exists x \in D_{i-1} : (x, y) \in R \wedge C(\sigma_i, x, y) = 1\}$;
  если $D_i = \varnothing$ **тогда**
    прервать цикл;
  конец
  $k^* \leftarrow i$;
конец
$Q(\mathcal{M}, \tau) \leftarrow \frac{k^*}{m}$;
если $k^* = m$ **тогда**
  возвратить k^*, $Q(\mathcal{M}, \tau)$, *τ полностью совместима с $\mathcal{M}$*;
конец
возвратить k^*, $Q(\mathcal{M}, \tau)$, *τ не полностью совместима с $\mathcal{M}$*;

Данный алгоритм является корректным в силу теорем 4.1 и 4.2. Если множество состояний X конечно, то алгоритм является конструктивно осуществимым, так как каждое множество D_i вычисляется за конечное число шагов.

Особый интерес представляет случай, когда $Q(\mathcal{M}, \tau) < 1$. Тогда полная трасса не совместима с моделью. Однако это не означает, что не существует ни одного совместимого префикса трассы. Напротив, существует максимальный индекс

$$k^*(\mathcal{M}, \tau) = \max \{i \in \{0, \ldots, m\} : D_i \neq \varnothing\},$$

после которого дальнейшее продолжение становится невозможным.

Смысл числа $k^*(\mathcal{M}, \tau)$ состоит в том, что первые $k^*(\mathcal{M}, \tau)$ наблюдений трассы ещё могут быть объяснены некоторым допустимым поведением модели, тогда как наблюдение $\sigma_{k^*(\mathcal{M},\tau)+1}$ уже не может быть согласовано ни с одним продолжением. Тем самым значение $k^*(\mathcal{M}, \tau)$ указывает на первый шаг трассы, на котором возникает расхождение между реализацией и моделью, а значение $Q(\mathcal{M}, \tau)$ характеризует степень близости трассы к полному согласованию.

Это наблюдение важно не только теоретически, но и практически, поскольку позволяет использовать предложенный метод не только для принятия или отклонения трассы в целом, но и для количественной оценки степени несоответствия и для локализации шага, на котором оно возникает.

5 Реализация задачи и решения средствами TLA+ и TLC

В предыдущих разделах задача верификации соответствия трассы программы и формальной спецификации была сформулирована как оптимизационная задача максимизации показателя качества согласованности модели и трассы. При этом полная согласованность трассы с моделью рассматривается как связанная с данной постановкой задача разрешимости, эквивалентная условию $Q(\mathcal{M}, \tau) = 1$. В настоящем разделе показывается, каким образом эта математическая постановка реализуется средствами языка TLA+ и TLC.

5.1 Общая схема реализации

Практическая реализация предложенного метода включает следующее:

а) **Спецификация алгоритма.** Она задаёт исходную формальную модель проверяемой системы: множество переменных состояния, начальное условие и отношение переходов [3].

б) **Спецификация трассы.** Она строится поверх базовой спецификации и добавляет ограничения, порождаемые наблюдаемой трассой выполнения программы [3].

в) **Конфигурация проверки TLC.** Она задаёт конкретные значения констант, подключает нужную спецификацию и определяет условия, при которых трасса считается принятой или отвергнутой [3].

г) **Библиотека для трассировки программ.** Она позволяет формировать трассы выполнения программы и объединять их в единую трассу при необходимости [4].

5.2 Соответствие модели и TLA+ спецификации

С точки зрения математической постановки модель задаётся системой переходов

$$\mathcal{M} = (X, X_0, R),$$

где X — множество состояний, X_0 — множество начальных состояний, а R — отношение переходов.

```
CONSTANT RM
VARIABLES rmState, tmState, tmPrepared, msgs
vars == << rmState, tmState, tmPrepared, msgs >>
Messages == [type : {"Prepared"}, rm : RM] \cup [type : {"Commit", "Abort"}]

TPInit ==
    /\ rmState = [r \in RM ⟼ "working"] /\ tmState = "init"
    /\ tmPrepared = {} /\ msgs = {}

RMPrepare(r) ==
    /\ rmState[r] = "working"
    /\ rmState' = [rmState EXCEPT ![r] = "prepared"]
    /\ msgs' = msgs \cup { [type ⟼ "Prepared", rm ⟼ r] }

RMRcvCommitMsg(r) ==
  /\ [type ⟼ "Commit"] \in msgs
  /\ rmState' = [rmState EXCEPT ![r] = "committed"]

TMRcvPrepared(r) ==
    /\ tmState = "init"
    /\ [type ⟼ "Prepared", rm ⟼ r] \in msgs
    /\ tmPrepared' = tmPrepared \cup {r}

TMCommit ==
    /\ tmState = "init" /\ tmPrepared = RM
    /\ tmState' = "done" /\ msgs' = msgs \cup { [type ⟼ "Commit"] }

(* RMRcvAbortMsg == ... *)
(* TMAbort == ... *)

TPNext ==
    \/ TMCommit \/ TMAbort
    \/ \E r \in RM :
         \/ RMPrepare(r)
         \/ TMRcvPrepared(r)

Spec == TPInit /\ [][TPNext]_vars

Consistent ==
    \A r1, r2 \in RM : ~(rmState[r1] = "aborted" /\ rmState[r2] = "committed")
```

Рисунок 1 – Спецификация алгоритма двухфазной транзакции

Для модуля `TwoPhase` (см. рис. 1) вектор состояния задаётся переменными

$$\texttt{vars} = \langle \texttt{rmState}, \texttt{tmState}, \texttt{tmPrepared}, \texttt{msgs} \rangle.$$

Следовательно, множество X в данном примере можно интерпретировать как множество всех допустимых значений указанной четвёрки переменных. Начальному множеству X_0 соответствует предикат `TPInit`, а отношению переходов R соответствует действие `TPNext`. Полная спецификация `Spec` задаёт множество допустимых бесконечных поведений, однако для цели валидации трасс существенны только конечные префиксы таких поведений, то есть именно те объекты, которые в математической постановке были обозначены через $\mathrm{B}(\mathcal{M})$.

Таким образом, исходная спецификация алгоритма TLA+ задает переходную систему, которая были введена в математической части.

5.3 Представление трассы программы

В математической постановке трасса программы задавалась последовательностью наблюдений $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$. В рассматриваемой реализации эта последовательность хранится во внешнем файле в формате NDJSON и считывается оператором `Trace` (см. рис. 2).

```
Trace ==
    IF "TRACE_PATH" \in DOMAIN IOEnv THEN
        ndJsonDeserialize(IOEnv.TRACE_PATH)
    ELSE
        Print(<<"TRACE_PATH environnement variable not found">>)
```

Рисунок 2 – Считывание трассы программы

Каждая строка такого файла представляет собой запись, описывающую один шаг исполнения программы. Запись может содержать имя события, его параметры и набор частичных обновлений логических переменных модели, пример такой записи представлен на рис 3.

```
{
  "clock": 4,
  "rmState": [
    { "op": "Update",
      "path": ["rm-0"],
      "args": ["prepared"] }
  ],
  "msgs": [
    { "op": "AddElement",
      "path": [],
      "args": [
        { "type":"Prepared", "rm":"rm-0" }
      ]
    }
  ],
  "event": "RMPrepare",
  "event_args": ["rm-0"]
}
```

Рисунок 3 – Пример наблюдения трассы

Тем самым множество наблюдений Σ в реализации соответствует множеству всех допустимых записей трассы данного формата. Последовательность `Trace` и есть конкретная реализация трассы τ.

5.4 Реализация предиката согласованности шага

Существенным свойством рассматриваемой реализации является синхронность сопоставления трассы и поведения модели. В математической постановке это выражено требованием, что каждой записи трассы соответствует ровно один переход согласованного поведения модели. В реализации это обеспечивается переменной `l` и оператором `IsEvent`, реализация которого представлена на рис. 4.

```
IsEvent(e) ==
    \* Equals FALSE if we get past the end of the log, causing checking to stop.
    /\ l \in 1..Len(Trace)
    /\ IF "event" \in DOMAIN logline THEN logline.event = e ELSE TRUE
    /\ l' = l + 1
    /\ UpdateVariables(logline)
```

Рисунок 4 – Реализация `IsEvent`

Оператор `IsEvent(e)` содержит следующие ограничения:

а) текущий индекс должен принадлежать диапазону $\texttt{l} \in 1..Len(\texttt{Trace})$;

б) имя события в текущей записи, если оно задано, должно совпадать с ожидаемым событием `e`;

в) после обработки текущей записи индекс увеличивается: $\texttt{l'} = \texttt{l} + 1$.

Следовательно, каждая успешно обработанная запись трассы продвигает систему на один шаг вперёд. Это и есть программная реализация условия

$$\forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1$$

из математической постановки. Иными словами, в текущем подходе одна запись трассы соответствует одному переходу ограниченной спецификации. Следует подчеркнуть, что речь идёт именно о переходе, а не о состоянии. Запись трассы интерпретируется как ограничение на пару соседних состояний модели.

В математической части предикат $C(\sigma, x, y)$ был введён абстрактно: он должен принимать значение 1 тогда и только тогда, когда наблюдение σ согласовано с переходом из состояния x в состояние y. В TLA+-реализации этот предикат не задаётся отдельной формулой, а реализуется через сочетание трёх механизмов (см. рис. 5:

а) проверки имени события;

б) проверки параметров события;

в) проверки и вычисления значений штрихованных переменных.

```
1  -------------------------- MODULE TwoPhaseTrace --------------------------
2  (***************************************************************************)
3  (* Trace spec that refines 2PC *)
4  (***************************************************************************)
5
6  EXTENDS TLC, Sequences, SequencesExt, Naturals, FiniteSets, Bags, Json,
7          IOUtils, TwoPhase, TVOperators, TraceSpec
8
9  (* Can be extracted from init *)
10 TPDefault(varName) ==
11     CASE varName = "rmState" -> [r \in RM |-> "working"]
12     []  varName = "tmState" -> "init"
13     []  varName = "tmPrepared" -> {}
14     []  varName = "msgs" -> {}
15
16 TPUpdateVariables(t) ==
17     /\
18         IF "rmState" \in DOMAIN t
19         THEN rmState' = UpdateVariable(rmState, "rmState", t)
20         ELSE TRUE
21     (*...*)
22
23 (* Predicate actions *)
24 IsTMCommit ==
25     /\ IsEvent("TMCommit")
26     /\ TMCommit
27
28 (* ... *)
29
30 TPTraceNext ==
31         \/ IsTMCommit
32         (* ... *)
33 ===========================================================================
```

Рисунок 5 – Обертка над спецификацией алгоритма для верификации трасс

Проверка имени события реализуется оператором `IsEvent`, описание которого приведено выше. Проверка параметров реализуется в действиях вида `IsTMCommit`, `IsRMPrepare`, `IsRMRcvCommitMsg` и других: если параметры заданы в трассе, то выбирается соответствующий экземпляр действия, а если не заданы, то допускается выбор любого допустимого параметра. Тем самым частичная информация, присутствующая в трассе, корректно интерпретируется как ограничение на множество допустимых переходов модели.

Проверка обновлений переменных реализуется оператором `TPUpdateVariables`. Он анализирует текущую запись трассы и, если для какой-либо переменной задан список обновлений, вычисляет соответствующее значение переменной при помощи общего оператора `ApplyUpdates`. Модуль `TVOperators` содержит универсальные операции `Update`, `AddElement`, `RemoveElement`, `SetKey` и другие, что позволяет интерпретировать записи трассы как частичные преобразования сложных структур данных.

Например, если запись трассы фиксирует, что в множество сообщений был добавлен элемент типа `Prepared`, а состояние некоторого ресурсного менеджера изменилось на `"prepared"`, то `TPUpdateVariables` накладывает ровно эти ограничения на значения `msgs'` и `rmState'`. Совместно с действием `RMPrepare(r)` это даёт точную реализацию условия согласованности данного наблюдения с переходом модели.

Таким образом, конкретная реализация предиката C зависит от выбранной спецификации и формата трассы, но общая схема остаётся неизменной: наблюдение задаёт ограничения на разрешённый шаг модели, а TLC проверяет, существует ли переход, удовлетворяющий этим ограничениям.

5.5 Ограниченная спецификация как реализация префиксной совместимости

В математической постановке для каждого $k \in \{0, \ldots, m\}$ были введены множества $\mathrm{B}_k(\mathcal{M}) \cap \mathrm{M}_k(\tau)$, состоящие из допустимых поведений модели длины k, согласованных с префиксом трассы $\tau^{(k)}$. Средствами TLA+ эти множества реализуются неявно через построение *ограниченной спецификации*, которая разрешает только те шаги, которые совместимы с текущей записью трассы.

В модуле `TraceSpec` такая ограниченная спецификация задаётся:

$$\mathtt{TraceSpec} \equiv \mathtt{TraceInit} \wedge [][\mathtt{TraceNext}]_{\langle \mathtt{l}, \mathtt{Vars} \rangle}.$$

Здесь `Vars` обозначает вектор переменных исходной спецификации, `BaseInit` — её начальный предикат, а `TraceNext` — дизъюнкция ограниченных действий, соответствующих шагам исходной модели.

Каждое из действий `TraceNext` соответствует некоторому действию исходной спецификации, но дополнительно ограничено текущей записью трассы. Поэтому поведения ограниченной спецификации представляют собой допустимые поведения исходной модели, согласованные с некоторым префиксом наблюдаемой трассы.

Иными словами, ограниченная спецификация реализует не только множество $\mathrm{B}_m(\mathcal{M}) \cap \mathrm{M}_m(\tau)$, соответствующее полной трассе, но и всю последовательность префиксных множеств

$$\mathrm{B}_k(\mathcal{M}) \cap \mathrm{M}_k(\tau), \qquad k \in \{0, \ldots, m\}.$$

Именно это делает возможным вычисление как факта полной совместимости, так и длины максимального совместимого префикса.

5.6 Соответствие рекуррентных множеств D_i и работы TLC

В математическом обосновании решения были введены множества

$$D_0, D_1, \ldots, D_m,$$

где D_i интерпретировалось как множество всех состояний модели, достижимых после согласования первых i наблюдений трассы. В реализации на TLC эти множества не вычисляются явно как отдельные математические объекты, однако вся процедура обхода ограниченной спецификации фактически эквивалентна именно такому вычислению.

Действительно, переменная `l` хранит номер текущей записи трассы. Начальное состояние ограниченной спецификации соответствует значению $\mathtt{l} = 1$, то есть ещё не обработано ни одной записи трассы. После обработки первой записи получаются состояния со значением $\mathtt{l} = 2$, после обработки второй — со значением $\mathtt{l} = 3$, и так далее. Поэтому множество всех достижимых состояний ограниченной спецификации с фиксированным значением

$$\mathtt{l} = i + 1$$

естественно соответствует множеству D_i из математической постановки.

Иначе говоря, TLC реализует пошаговое построение множеств D_i не в явном формульном виде, а через стандартный перебор достижимых состояний ограниченной спецификации. При этом состояния с фиксированным значением $\mathtt{l} = i + 1$ соответствуют множеству D_i.

Следовательно, обход ограниченной спецификации средствами TLC позволяет определить не только факт полной согласованности трассы, но и длину максимального согласованного префикса

$$k^*(\mathcal{M}, \tau) = \max\{\, i \in \{0, \ldots, m\} : D_i \neq \varnothing \,\}.$$

На этой основе вычисляется и показатель качества

$$Q^*(\mathcal{M}, \tau) = \frac{k^*(\mathcal{M}, \tau)}{m}.$$

5.7 Критерий успешной проверки в TLC

В математической постановке основной результат задаётся показателем качества $Q^*(\mathcal{M}, \tau) = \frac{k^*(\mathcal{M},\tau)}{m}$, где $k^*(\mathcal{M}, \tau)$ — длина максимального согласованного префикса трассы. В реализации на TLC эта величина извлекается из диаметра построенного ограниченного пространства состояний.

Поскольку в ограниченной спецификации при каждом шаге переменная `l` увеличивается на единицу, диаметр графа состояний соответствует максимальной длине согласованного префикса с точностью до единицы. Если обозначить через d значение `TLCGet("stats").diameter`, то

$$k^*(\mathcal{M}, \tau) = d - 1.$$

```
1 TraceAccepted ==
2     LET d == TLCGet("stats").diameter IN
3     IF d - 1 = Len(Trace) THEN TRUE
4     ELSE Print(<<"Failed matching the trace to (a prefix of) a behavior:", Trace[d],
5                  "TLA+ debugger breakpoint hit count " \o ToString(d+1)>>, FALSE)
```

Рисунок 6 – Реализация `TraceAccepted`

Оператор `TraceAccepted` (см.рис. 6) реализует не всю оптимизационную постановку, а связанную с ней задачу разрешимости. Он проверяет условие

$$d - 1 = \mathrm{Len}(\mathrm{Trace}),$$

то есть эквивалентное условию

$$Q^*(\mathcal{M}, \tau) = 1.$$

Если это условие выполняется, трасса полностью согласована с моделью. Если же оно нарушается, то полная согласованность отсутствует, однако по значению диаметра всё равно может быть определён максимальный согласованный префикс и, следовательно, показатель качества согласованности.

Использование диаметра оправдано структурой ограниченной спецификации: при каждом шаге переменная `l` увеличивается на единицу, поэтому обход пространства состояний естественным образом организован по уровням, соответствующим длине уже согласованного префикса трассы. Следовательно, максимальный достигнутый уровень эквивалентен длине максимального согласованного префикса трассы, а достижение последнего уровня эквивалентно случаю полной согласованности.

5.8 Особенности и ограничения рассматриваемой реализации

Следует отметить, что реализованная схема соответствует той математической постановке, которая была принята в работе, а именно синхронному сопоставлению трассы и поведения модели. Каждой записи трассы соответствует ровно один переход ограниченной спецификации. Это упрощает как математическое описание, так и программную реализацию.

Основное ограничение связано с неполнотой трассы. Если записи трассы содержат слишком мало информации, то множество допустимых интерпретаций отдельного шага становится слишком широким. Тогда TLC может найти некоторое согласованное поведение модели, хотя фактическое поведение программы восстановлено по трассе неоднозначно. Таким образом, практическая точность метода зависит не только от корректности спецификации, но и от качества инструментирования кода и полноты логируемых данных.

ЗАКЛЮЧЕНИЕ

В работе рассмотрена задача верификации совместимости трасс исполнения программы с формальной спецификацией, заданной на языке TLA+. Показано, что данная задача возникает при необходимости поддерживать соответствие между развивающейся программной реализацией и её формальной моделью.

В ходе работы была предложена методика верификации, основанная на инструментировании программной реализации, построении трассы наблюдаемых шагов исполнения и её последующей интерпретации как системы ограничений на переходы формальной модели.

Основным теоретическим результатом работы стала строгая математическая постановка задачи в терминах системы переходов, трассы наблюдений и предиката согласованности шага. Показано, что задача допускает оптимизационную интерпретацию, в которой максимизируется показатель качества согласованности, эквивалентный максимизации длины совместимого префикса трассы, а полная совместимость трассы с моделью рассматривается как связанная с этой постановкой задача разрешимости.

Для решения поставленной задачи введена последовательность множеств достижимых состояний, согласованных с префиксами трассы и достижимых по допустимым переходам модели. Доказано, что каждое такое множество содержит в точности те состояния модели, которые достижимы после согласования соответствующего префикса наблюдений. На этой основе получен критерий нахождения максимального совместимого префикса, введён показатель качества согласованности трассы с моделью и построен алгоритм решения задачи.

Практическая часть работы показала, каким образом полученная математическая постановка реализуется средствами TLA+ и TLC. Рассмотрено соответствие между системой переходов и TLA+-спецификацией, между наблюдаемой трассой и ограниченной спецификацией, а также между рекуррентным построением множеств достижимых состояний и обходом пространства состояний, выполняемым TLC.

Таким образом, цель работы достигнута: разработана методика верификации совместимости трассы исполнения программы с TLA+ спецификацией, сформулирована её математическая постановка, обосновано решение и показана его реализация средствами TLA+ и TLC.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ПОЛОЖЕНИЕ №03.02-06/2. О структурном подразделении Научно-учебный комплекс «Информатика и системы управления». — Взамен Положения о структурном подразделении №03.02-06/1 от 12.04.2012. — МГТУ им. Н.Э. Баумана, 2022. — 8 с.

2. Keller R. M. Formal Verification of Parallel Programs // Communications of the ACM. — 1976. — Июль. — Т. 19, № 7. — С. 371—384.

3. Железцов Н. two-phase-tlac: реализации алгоритма двухфазного коммита для валидации соответствия TLA+ спецификации. — 2025. — URL: https://github.com/Serpentian/two-phase-tlac (дата обращения 15.11.2025).

4. Железцов Н. tlac: C библиотека для трассировки кода. — 2025. — URL: https://github.com/Serpentian/tlac (дата обращения 15.09.2025).

ПРИЛОЖЕНИЕ А

Методика верификации

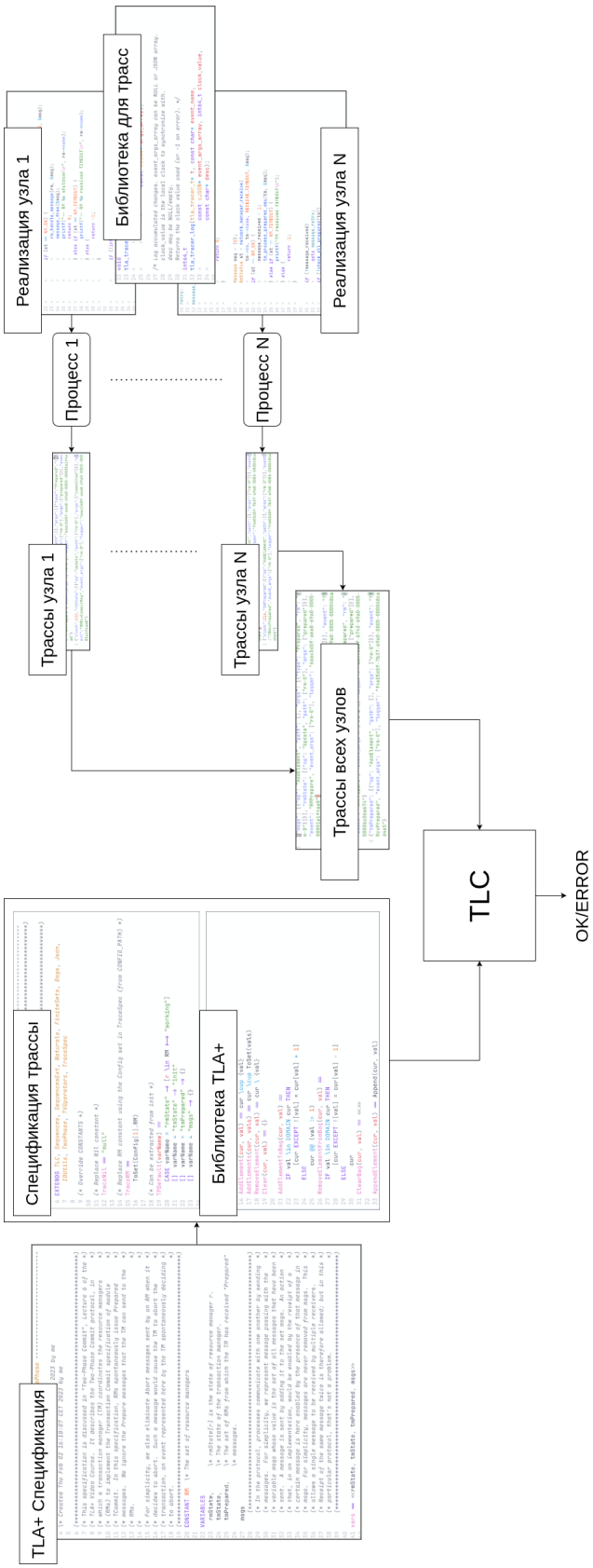

Рисунок А.1 – Методика верификации